Industrial Internship Report on

"BankingInformationSystem"

Prepared by

Mayur Tayade

Executive Summary
This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time. My project was BankingInformationSystem – a Core Java console application using OOP. It models a bank account as a class (BankAccount) with deposit, withdrawal, and display methods, driven by a Scanner menu loop in the main class. This internship gave me a very good opportunity to get exposure to Industrial problems and design/implement solution for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface 4
2 Introduction 6
2.1 About UniConverge Technologies Pvt Ltd 6
2.2 About upskill Campus....10
2.3 Objective 12
2.4 Reference 12
2.5 Glossary....12
3 Problem Statement....13
4 Existing and Proposed solution 14
5 Proposed Design/ Model 15
5.1 High Level Diagram (if applicable) 15
5.2 Low Level Diagram (if applicable)....15
5.3 Interfaces (if applicable)....15
6 Performance Test 17
6.1 Test Plan/ Test Cases 17
6.2 Test Procedure....18
6.3 Performance Outcome....18
7 My learnings....19
8 Future work scope 20

1 Preface

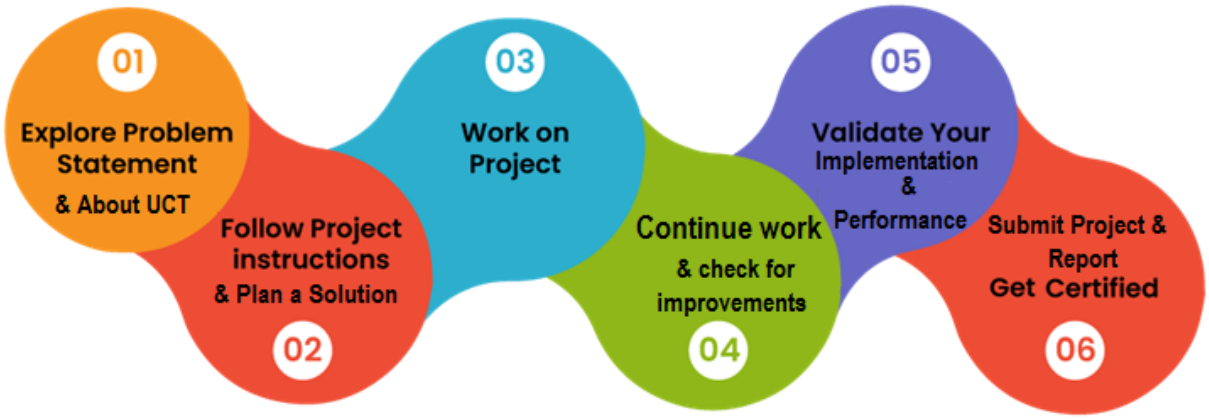
This internship spanned 4 weeks (January–February 2026) in Core Java. Week 1: Java history, Hello World, javac/java commands. Week 2: Platform independence (JVM, Bytecode, JDK, JRE), 10-question quiz. Week 3: eBook study (David J. Eck) – variables, data types, operators, I/O. Week 4: Advanced OOP (Interfaces, Method Overloading, Abstract Classes), 9-mark quiz, and development of the BankingInformationSystem project.

A relevant internship bridges the gap between academic learning and industry practice. This Core Java internship gave exposure to structured software development, OOP design, and professional coding standards – directly valuable for placements and a Java developer career.

The BankingInformationSystem is a Java console application built using Core Java and OOP. A BankAccount class encapsulates account data (account number, name, balance) and operations (deposit, withdraw, display). The main class manages user interaction via Scanner and a do-while menu loop.

I am grateful to UpSkill Campus (USC) and UniConverge Technologies Pvt Ltd (UCT) for providing this internship. UCT supplied the problem statement and mentor support throughout all 4 weeks.

Week 1 – Java orientation & Hello World. Week 2 – Platform independence & quiz. Week 3 – eBook-based fundamentals study. Week 4 – Advanced OOP concepts, quiz, and BankingInformationSystem project development.



I gained strong hands-on knowledge of Core Java and OOP. Designing the BankAccount class taught me encapsulation in practice. Weekly quizzes reinforced conceptual clarity, and completing the project built genuine coding confidence.

I sincerely thank the team at USC and UCT for this opportunity. I also thank the faculty at Nutan Maharashtra Institute of Engineering & Technology, Talegaon Dabhade, for their continuous encouragement.

To my juniors: OOP concepts become truly clear only when applied in a real project. Be consistent, practice coding daily, and make the most of every internship opportunity. The discipline you build now will shape your entire career.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.

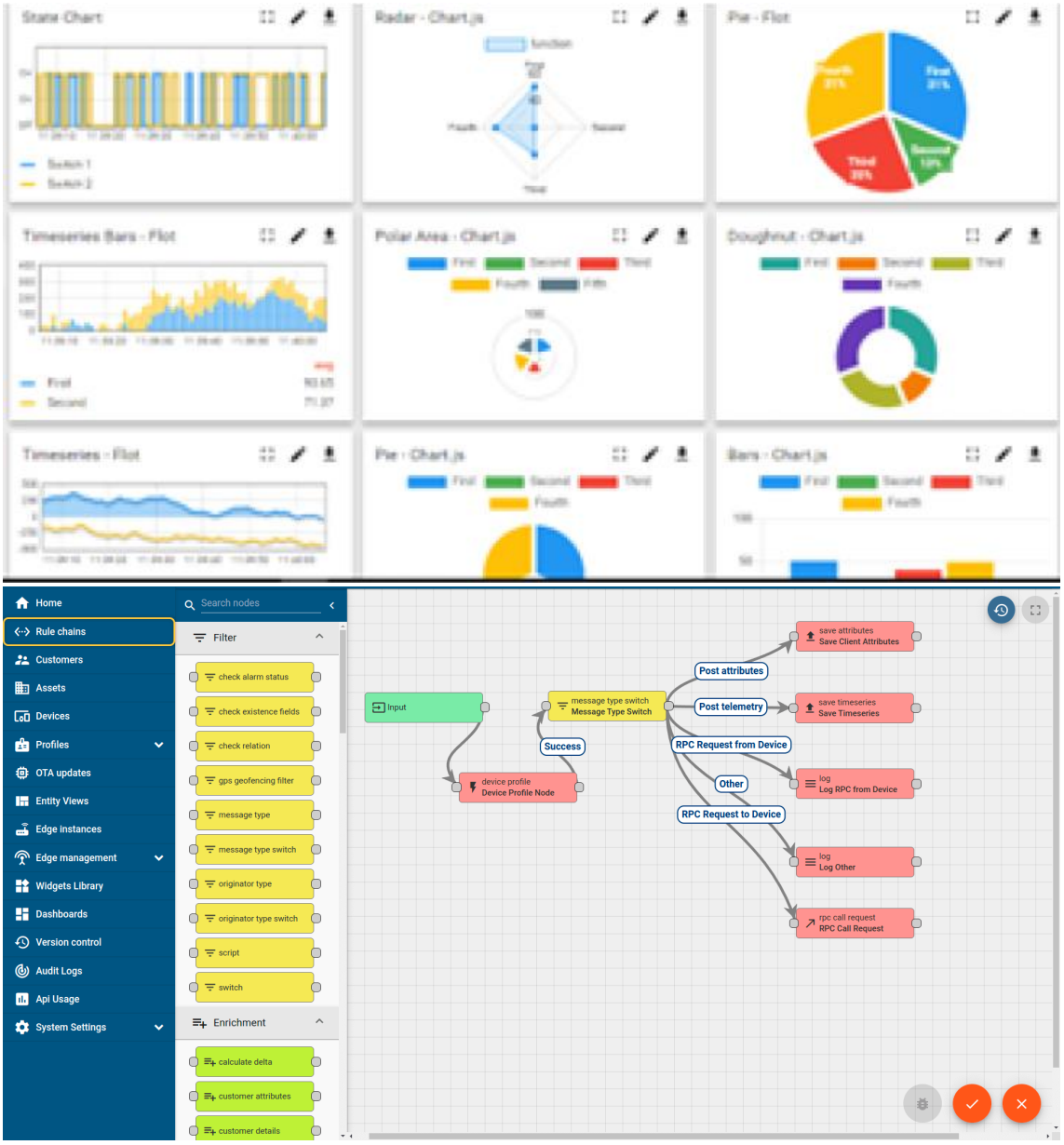
i. UCT IoT Platform (uct Insight)

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine



ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.

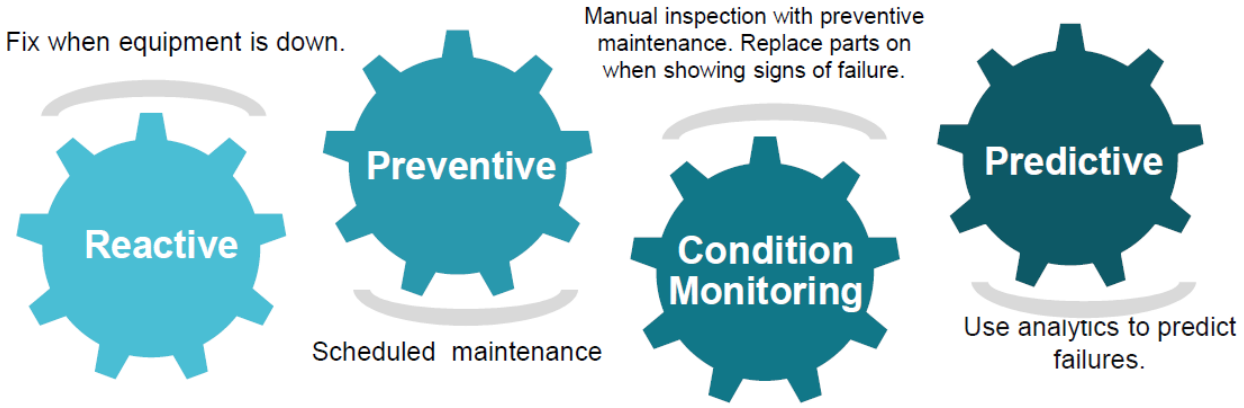
Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

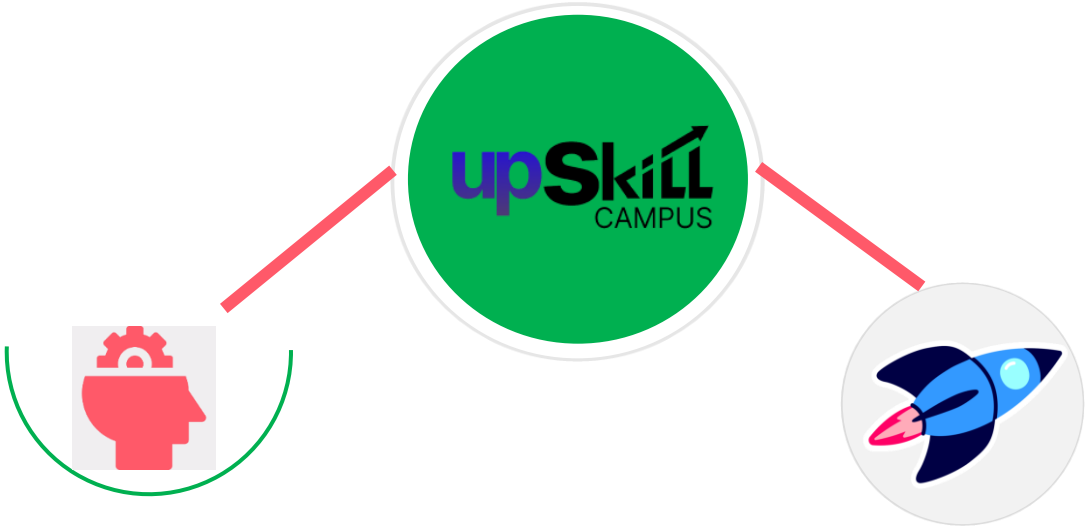
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

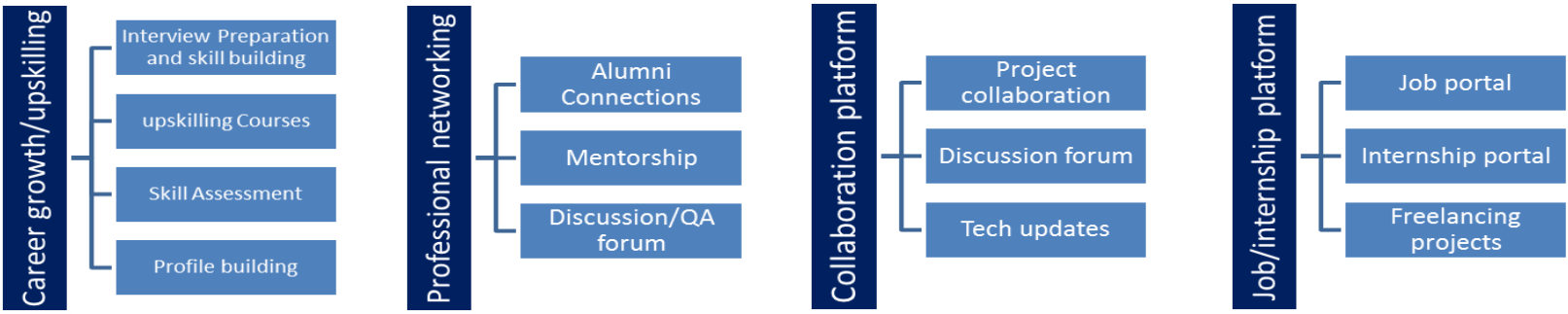
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to

- ☛ get practical experience of working in the industry.
- ☛ to solve real world problems.
- ☛ to have improved job prospects.
- ☛ to have Improved understanding of our field and its applications.
- ☛ to have Personal growth like better communication and problem solving.

2.5 Reference

[1] David J. Eck, "Introduction to Programming Using Java", 9th Ed.

[2] Oracle Java SE Documentation – https://docs.oracle.com/javase/

[3] UpSkill Campus – https://www.upskillcampus.com/

2.6 Glossary

Terms	Acronym

3 Problem Statement

In the assigned problem statement, the task was to build a BankingInformationSystem using Core Java and OOP concepts learned during the internship.

Design a Java console application that simulates core banking operations: (1) Create a bank account with account number, holder name, and initial balance. (2) Deposit – add an amount to the balance. (3) Withdraw – deduct an amount if sufficient balance exists; otherwise print "Insufficient Balance". (4) Display – show account number, name, and current balance. (5) A menu-driven interface using Scanner with a do-while loop to let the user repeatedly choose an operation until they select Exit. The design must apply OOP: encapsulate all data and operations inside a BankAccount class, and separate UI logic (main class) from business logic (BankAccount class).

4 Existing and Proposed solution

Procedural Java programs that handle banking in a single main() method mix data and logic together, making code hard to maintain or extend. They typically have no structured balance validation (risk of negative balance) and no reusable components – every new feature requires rewriting the entire program. Real manual teller systems are slow, error-prone, and require physical presence.

The BankingInformationSystem uses an OOP design: a BankAccount class (constructor + 3 methods) encapsulates all account data and logic. The main class (BankingInformationSystem) handles only user input/output via Scanner, keeping concerns separated. This mirrors professional software design where real-world entities are modelled as classes.

(1) Encapsulation – account fields modified only through controlled methods. (2) Balance validation – withdraw() prevents negative balances via an if-else check. (3) Modularity – BankAccount class is self-contained and reusable (can instantiate multiple accounts). (4) Robustness – invalid menu choices handled by a default case without crashing.

4.1 Code submission (Github link)

upskillcampus/BankingInformationSystem.java at main · mayurTayade2006/upskillcampus

4.1.1 Report submission (Github link) : first make placeholder, copy the link.

upskillcampus/BankingInformationSystem_Mayur_USC_UCT.pdf at main · mayurTayade2006/upskillcampus

5 Proposed Design/ Model

The BankingInformationSystem has a straightforward two-layer flow: START – main() prompts for account number, name, initial balance; instantiates BankAccount object. MENU LOOP – do-while displays: 1.Deposit 2.Withdraw 3.Display Account 4.Exit. DEPOSIT (case 1) – reads amount, calls account.deposit(d); balance += amount. WITHDRAW (case 2) – reads amount, calls account.withdraw(w); if balance >= w: balance -= w, else prints "Insufficient Balance". DISPLAY (case 3) – calls account.display(); prints accNo, name, balance. EXIT (case 4) – prints "Thank You"; do-while condition (choice != 4) ends loop. DEFAULT – prints "Invalid Choice"; loop continues.

5.1 High Level Diagram (if applicable)

Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram (if applicable)

5.3 Interfaces (if applicable)

Not applicable. BankingInformationSystem is a single-class console application with no external interfaces, hardware, or network protocols.

6 Performance Test

This is very important part and defines why this work is meant of Real industries, instead of being just academic project.

Key constraints identified: (1) Balance Accuracy – balance must never go negative. (2) Input Handling – menu must not crash on invalid input. (3) Menu Loop – loop must continue correctly after every valid and invalid operation.

(1) Balance Accuracy: withdraw() uses if(balance >= amount) check before deducting – balance is never reduced below zero. (2) Input Handling: Scanner reads the correct types (nextInt/nextDouble) per field; the default: switch case catches unexpected menu choices. (3) Loop Continuity: do-while with condition (choice != 4) ensures the menu always reappears after each operation until the user explicitly exits.

All constraints validated – see Test Plan/Outcome below.

For this console application the primary constraint is correctness of balance logic and menu robustness, not hardware-level metrics.

All constraints were tested successfully. For future production use: add try-catch for InputMismatchException to handle non-numeric input gracefully; enforce minimum deposit amount; replace in-memory balance with JDBC + MySQL for data persistence.

6.1 Test Plan/ Test Cases

TC-01: Deposit – Input amount 500. Expected: balance +500, prints "Deposit Successful". Result: PASS

TC-02: Withdraw valid – Balance 1000, withdraw 400. Expected: balance 600, "Withdrawal Successful". Result: PASS

TC-03: Withdraw insufficient – Balance 200, withdraw 500. Expected: "Insufficient Balance", balance unchanged. Result: PASS

TC-04: Display – After operations. Expected: correct accNo, name, balance printed. Result: PASS

TC-05: Invalid menu choice – Enter 9. Expected: "Invalid Choice", menu reappears. Result: PASS

TC-06: Exit – Enter choice 4. Expected: "Thank You", program exits. Result: PASS

6.2 Test Procedure

Step 1 – Compile: javac BankingInformationSystem.java. Verify zero errors.

Step 2 – Run: java BankingInformationSystem. Input accNo: 1001, Name: Mayur Tayade, Balance: 1000.

Step 3 (TC-01) Deposit: Select 1, enter 500. Verify "Deposit Successful".

Step 4 (TC-02) Withdraw valid: Select 2, enter 400. Verify "Withdrawal Successful".

Step 5 (TC-03) Withdraw insufficient: Select 2, enter 5000. Verify "Insufficient Balance".

Step 6 (TC-04) Display: Select 3. Verify accNo, name, balance match expected values.

Step 7 (TC-05) Invalid: Enter 9. Verify "Invalid Choice" and menu reappears.

Step 8 (TC-06) Exit: Enter 4. Verify "Thank You" and program terminates.

6.3 Performance Outcome

All 6 test cases passed. Key observations:

- Deposit correctly incremented balance in all test runs.
- Withdrawal with sufficient balance deducted correctly; balance never went negative.
- "Insufficient Balance" displayed correctly when withdrawal exceeded balance.
- Display output matched all entered and computed values accurately.
- Invalid menu choice handled without crash; loop continued normally.
- Program exited cleanly on choice 4 with "Thank You" confirmation.

7 My learnings

Core Java: Mastered syntax, data types, operators, control flow, and the Java compilation process (javac, java). OOP: Learned to design classes, create objects, write constructors and instance methods. Applied Encapsulation (BankAccount class bundles data + behavior). Advanced OOP (Week 4): Understood Interfaces, Method Overloading, and Abstract Classes and how they enable abstraction and polymorphism. Java Architecture: Understood JVM, Bytecode, JDK, JRE, and why Java is platform-independent. Problem Solving: Weekly quizzes (10-mark, 9-mark) sharpened logical thinking. Career Impact: These skills form the foundation of enterprise Java development and support placements and a career as a Java Software Developer.

8 Future work scope

1. Multiple Accounts – Use ArrayList to manage multiple BankAccount objects. 2. Inheritance – Add SavingsAccount and CurrentAccount subclasses with type-specific rules. 3. Interfaces – Define a BankingOperations interface applied across all account types. 4. Exception Handling – Replace print statements with try-catch and custom exceptions. 5. File/DB Persistence – Use File I/O or JDBC+MySQL to save data permanently. 6. GUI – Rebuild as a Java Swing or JavaFX desktop application.